Hybrid Unsupervised–Supervised Document Coherence Verification Using Restricted Boltzmann Machines and Semantic Entity Graphs

Molly Maskrey

July 2026

Abstract

Consulting and analytical deliverables — PowerPoint presentations, PDF reports, and structured documents — are subject to a class of coherence errors that evade traditional proofreading: implicit cross-document assertions that are spatially separated from the evidence that should support them. A cover slide bearing a 2023 date may contain charts and data tables reflecting 2022 vintage data; a client name may drift across slides following an acquisition; numerical claims may be inconsistent with the supporting tables from which they were derived. These errors are invisible to linear proofreading because they require the reader to maintain a running ledger of assertions across the full document context — a working memory demand that human proofers do not reliably satisfy.

We propose a hybrid document coherence verification architecture combining unsupervised Restricted Boltzmann Machine (RBM) anomaly detection over a semantic entity coherence graph with a supervised semantic contradiction classifier for higher-order linguistic inconsistencies. The unsupervised component requires no labeled error examples and learns the joint distribution of entity relationships in well-formed documents; anomalies emerge as low- probability states under the learned model. The supervised component handles semantic contradiction cases — claims inconsistent with supporting evidence, bullet points contradicting their own slide headers — that require language understanding beyond co-occurrence statistics. Structural extraction leverages Hugging Face LayoutLMv3 for layout-aware document understanding. The architecture is designed for deployment within an air-gapped enterprise Azure environment using open-source language models hosted on Azure Synapse Analytics and Azure Batch compute, satisfying SOC 2 Type II data governance requirements that prohibit client document content from transiting external API endpoints.

1 Introduction

Document quality assurance in consulting and analytical practice presents a specific and underserved problem class. Traditional proofreading tools — spelling checkers, grammar engines, style linters — operate on local syntactic features and are blind to the semantic coherence relationships that span document sections, slides, and structural elements. The error class of concern here is fundamentally different: it arises not from malformed sentences but from implicit assertions distributed across the document that are inconsistent with one another.

Consider a PowerPoint deliverable in which the cover slide reads "2023 Medicaid Enrollment Trends" while the embedded data tables contain values through Q4 2022 only. Neither the cover nor the data table contains an error in iso-

lation. The inconsistency is a property of the *relationship* between them — a relationship that a human proofer reading linearly does not naturally reconstruct. The same failure mode applies to client name drift following corporate acquisitions, drug indication language that evolves across slides, percentage claims that round differently in narrative text versus supporting tables, and temporal references that accumulate across a document without being reconciled.

These are, fundamentally, working memory problems. The document carries implicit assertions that require the reader to hold prior context while evaluating later content. Automated verification systems that model the document as a coherence graph — where nodes are semantic entities and edges represent cross-reference relationships — can perform this ledger-keeping reliably and at scale.

We present a hybrid architecture that addresses this problem in two layers. The first layer applies an unsupervised Restricted Boltzmann Machine to a semantic entity graph extracted from the document, learning the joint distribution of entity co-occurrence patterns in well-formed documents and flagging configurations that are improbable under that distribution. The second layer applies a supervised semantic contradiction classifier to sentence- and bullet-level content pairs where claims and supporting evidence may be linguistically inconsistent in ways that entity co-occurrence statistics do not capture.

The system is designed explicitly for deployment within an enterprise Azure environment using locally hosted open-source language models, satisfying SOC 2 Type II data governance requirements that prohibit client document content from transiting external API boundaries.

2 Background and Related Work

2.1 Document Coherence and Consistency

Coherence in document understanding has been studied extensively in the context of discourse structure [1], coreference resolution [2], and information extraction [3]. However, the cross-document and cross-structural consistency problem we address — entity assertion coherence across heterogeneous document elements including slide titles, chart labels, data tables, and footnotes — has received comparatively little attention in the context of business document quality assurance.

2.2 Layout-Aware Document Understanding

Raw text extraction from structured documents discards the spatial and visual layout relationships that are essential for understanding document structure. LayoutLM [4] and its successor LayoutLMv3 [5] jointly encode text tokens, their bounding box positions, and visual features from rendered page images, enabling document understanding models to distinguish structurally distinct elements — headline claims versus footnote qualifications, chart titles versus table cells — that carry identical lexical content but differ in their structural role and therefore their semantic weight in a coherence analysis.

2.3 Restricted Boltzmann Machines for Anomaly Detection

Restricted Boltzmann Machines [6,7] are energy-based generative models that learn a probability distribution over visible input configurations via a latent hidden layer. The RBM's energy function over visible units $\mathbf{v}$ and hidden units $\mathbf{h}$ is:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h} - \mathbf{v}^\top \mathbf{W} \mathbf{h} \qquad (1)$$

where $\mathbf{W}$ is the weight matrix, $\mathbf{b}$ and $\mathbf{c}$ are visible and hidden bias vectors respectively. The joint probability of a configuration is:

$$P(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} e^{-E(\mathbf{v},\mathbf{h})} \tag{2}$$

where Z is the partition function. Anomaly detection using RBMs exploits the reconstruction error: inputs that are inconsistent with the learned distribution produce high reconstruction error when the model attempts to regenerate them from their latent representation [8].

The key advantage for our application is that RBMs require only clean, well-formed documents for training. Labeled error examples are not required. Anomalies emerge as out-of-distribution configurations relative to the learned manifold of coherent documents.

2.4 Graph-Based Document Representation

Network analysis of document structure has been applied to summarization [9], information retrieval [10], and knowledge graph construction [11]. The application of network centrality measures to identify semantically critical versus peripheral entities within a document provides a natural framework for consistency checking: entities that occupy high-centrality positions in a coherent document should exhibit consistent co-occurrence patterns across all structural positions in which they appear.

3 Architecture

3.1 System Overview

The proposed system operates in two sequential layers as illustrated in Figure 1. Layer 1 performs unsupervised entity coherence graph analysis using an RBM. Layer 2 applies supervised semantic contradiction detection to flagged content pairs. A layout-aware document ingestion pipeline upstream of both layers handles structural extraction from PowerPoint and PDF sources using LayoutLMv3.

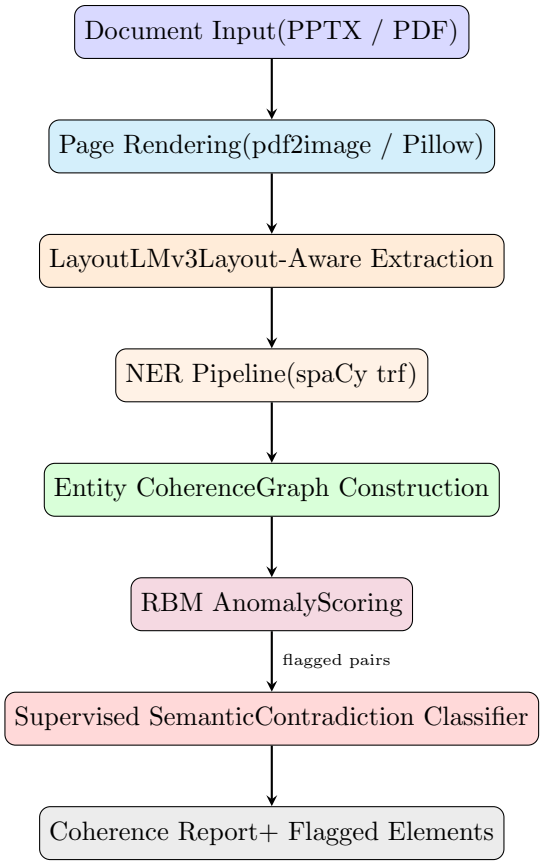


Figure 1: System architecture for hybrid document coherence verification with LayoutLMv3 structural extraction.

3.2 Document Ingestion and Layout-Aware Structural Extraction

The structural extraction pipeline operates in two stages. First, document pages are rendered to images: `python-pptx` with `Pillow` handles PPTX rendering; `pdf2image` handles PDF rendering. The rendered page images, combined with text extracted via `python-pptx` element iteration or Azure Document Intelligence, are passed to LayoutLMv3-large as input.

LayoutLMv3 [5] encodes each text token jointly with its normalized bounding box coordinates (x_0, y_0, x_1, y_1) and the visual patch features from the rendered page image. This trimodal encoding allows the model to assign structurally differentiated representations to tokens that share lexical content but differ in document position — distinguishing a year appearing in

a slide title from the same year appearing in a table footnote, a distinction that is invisible to text-only extraction.

The LayoutLMv3 output for each token is a contextualized embedding that encodes both semantic content and structural role. A lightweight classification head trained on document element types assigns each extracted span to a structural category:

- **Title**: slide or section heading
- **Body**: narrative bullet or paragraph text
- **Chart title / axis label**: chart annotation
- **Table cell**: tabular data value or header
- **Footnote / source**: citation or qualification
- **Speaker note**: supplementary non-displayed text

Named entity recognition is then applied to the structurally classified spans using `spaCy en_core_web_trf`, producing entity tuples (e, t, s, p) where e is the entity value, t is the entity type, s is the slide or page index, and p is the structural position class assigned by LayoutLMv3.

3.3 Entity Coherence Graph Construction

A weighted undirected graph $G = (V, E)$ is constructed over the extracted entity set. Nodes V represent unique entity values within the document. Edges E represent co-occurrence relationships between entities, weighted by co-occurrence frequency and structural relationship type.

Edge weight w_{ij} between entities e_i and e_j is defined as:

$$w_{ij} = \sum_{k} \alpha_k \cdot \mathbb{1}[(e_i, e_j) \in C_k] \tag{3}$$

where C_k is the set of co-occurrence pairs at structural level k and α_k is a structural weight coefficient with $\alpha_{\text{same-element}} > \alpha_{\text{same-slide}} > \alpha_{\text{cross-slide}}$.

Node centrality is computed using degree centrality, betweenness centrality, and eigenvector centrality. The 2022/2023 year inconsistency manifests as a connectivity anomaly: "2023" occupies a high-centrality node across title and body structural positions while "2022" appears as a weakly connected outlier concentrated in table cell positions.

3.4 Layer 1: RBM Anomaly Detection

The entity coherence graph is featurized into a fixed-dimensional visible layer input vector $\mathbf{v}$ encoding entity centrality scores by type, co-occurrence pattern vectors for each entity type pair, connectivity profiles per entity, and entity type homogeneity within co-occurrence clusters.

The RBM is trained on a corpus of clean consulting documents using contrastive divergence [12]. At inference time, the anomaly score is:

$$\mathcal{A}(\mathbf{v}) = \|\mathbf{v} - \hat{\mathbf{v}}\|^2 \tag{4}$$

where $\hat{\mathbf{v}}$ is the one-step Gibbs reconstruction. Per-entity reconstruction error contributions localize the anomaly to specific entity relationships.

3.5 Layer 2: Supervised Semantic Contradiction Detection

Entity-level anomalies flagged by Layer 1 are passed to a supervised classifier operating on content pairs (c_i, c_j), outputting:

$$y \in \{\texttt{CONSISTENT}, \texttt{CONTRADICTS}, \texttt{UNVERIFIABLE}\} \tag{5}$$

with calibrated confidence per class. The classifier is a fine-tuned Phi-3-medium-128k instance served from an Azure ML managed endpoint.

4 Azure Cloud Implementation

4.1 Compliance Requirements

Forte Analytics maintains SOC 2 Type II certification. The deployment architecture satisfies the following controls:

- **CC6.1** — All model inference executes within Forte's Azure tenant; no document content transits external network boundaries
- **CC6.6** — All internal service communication encrypted via TLS 1.3 within the virtual network
- **CC7.2** — All document processing events logged to Azure Monitor with immutable audit trail
- **C1.1** — Client document content processed only within tenant-controlled compute; not retained beyond session scope
- **PI1.4** — Model inference results traceable to pinned model versions via Azure ML model registry

4.2 Compute Strategy: Synapse and Azure Batch

The workload decomposes naturally into two compute regimes that map onto distinct Azure services.

Batch corpus processing — building and periodically refreshing the RBM training corpus by running LayoutLMv3 extraction over Forte's archive of clean historical documents — is an embarrassingly parallel workload over independent documents. Azure Batch is the appropriate service for this regime. Jobs are submitted as plain Python task scripts; Azure Batch distributes them across a pool of GPU-enabled nodes (NC-series) without requiring Spark or any distributed computing abstraction. Each task processes one document and writes its output — entity tuples and graph adjacency representation — to Azure Blob Storage. No PySpark code is required.

Pipeline orchestration and intermediate data management — coordinating the multi-step inference pipeline for individual document submissions, joining extraction outputs with graph construction results, and managing the data flow between services — is handled by Azure Synapse Analytics pipelines. Synapse pipelines provide a visual orchestration layer over Azure services without requiring Spark DataFrames; activities invoke Azure ML endpoints, Azure Batch jobs, and Azure Functions directly. Where distributed data processing is required — for example, computing graph centrality metrics over a large accumulated entity corpus — Synapse Spark pools are available with the `pyspark.pandas` API surface, which provides a pandas-compatible interface over distributed compute without native PySpark DataFrame syntax.

4.3 Open-Source Model Selection

Table 1 summarizes the open-source model stack.

4.4 Infrastructure Architecture

Figure 2 illustrates the full Azure deployment architecture. All components reside within a single Azure Virtual Network with no public endpoints for document content.

4.5 Data Flow and Retention

Documents submitted for coherence verification follow a controlled data flow:

1. Document uploaded to Azure Blob Storage via internal API Management endpoint; encrypted at rest with customer-managed keys (CMK) in Azure Key Vault
2. Azure Synapse pipeline triggered on Blob Storage event; orchestrates downstream processing steps
3. LayoutLMv3 extraction and NER run as Azure Batch tasks on GPU and CPU pools respectively; outputs written as structured JSON to Blob Storage

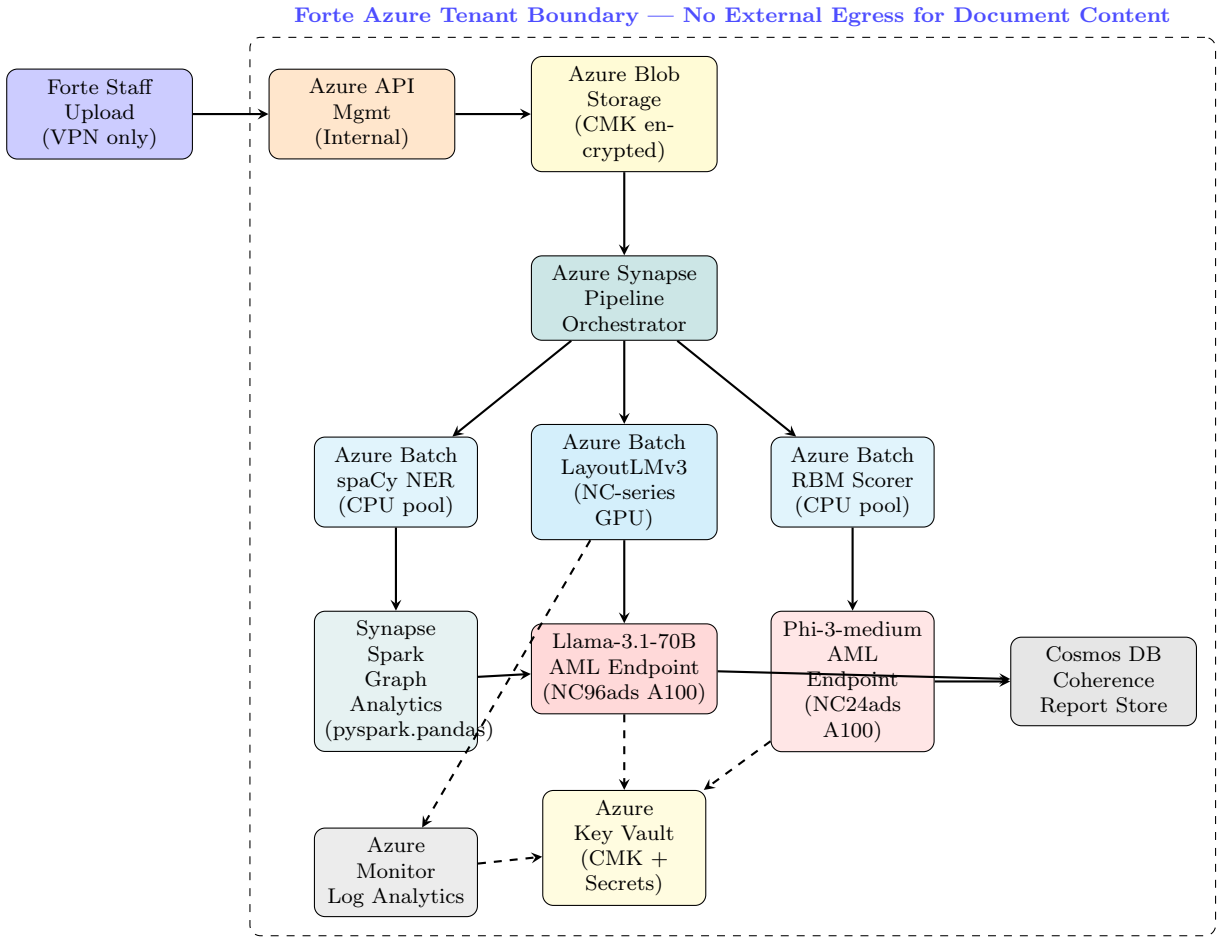


Figure 2: Azure deployment architecture. Azure Synapse pipelines orchestrate the full processing workflow. Azure Batch handles embarrassingly parallel corpus jobs (LayoutLMv3, NER, RBM scoring) in plain Python without Spark. Synapse Spark pools with `pyspark.pandas` handle graph analytics requiring distributed compute. AML managed endpoints serve Llama-3.1-70B and Phi-3-medium within the tenant boundary. All document content remains within Forte's Azure tenant at all times.

4. Graph construction and centrality computation run on Synapse Spark pool via `pyspark.pandas`; RBM anomaly scoring run as Azure Batch CPU task

5. Flagged content pairs passed to AML managed endpoints for Llama-3.1-70B claim extraction and Phi-3-medium contradiction classification

6. Coherence report written to Cosmos DB; raw document and all intermediate representations purged from Blob Storage and compute node memory at pipeline completion

7. All processing events written to Azure Monitor Log Analytics with 90-day immutable retention for SOC 2 audit support

Document content is never written to any persistent storage layer beyond the initial encrypted Blob Storage intake. Intermediate representations are processed in-memory within Azure Batch task boundaries or Synapse Spark executor memory and are not persisted independently.

Component	Model	Compute
Layout extraction	LayoutLMv3-large (HF)	Azure Batch NC-series GPU
NER	spaCy trf	Azure Batch CPU pool
Structural NLP	Llama-3.1-70B -Instruct	AML Managed NC96ads A100
Contradiction	Phi-3-medium -128k	AML Managed NC24ads A100
Embeddings	E5-large-v2	Azure Batch CPU pool
RBM scoring	Custom PyTorch	Azure Batch CPU pool
Graph analytics	NetworkX / cuGraph	Synapse Spark GPU pool

Table 1: Open-source model stack, compute assignments, and hosting services.

4.6 Model Governance

Open-source models are registered in the Azure ML model registry with pinned version hashes. LayoutLMv3 and E5-large-v2 are loaded from the Hugging Face Hub on first deployment and subsequently served from Azure Container Registry, eliminating runtime dependency on external model registries. Model updates require explicit registry promotion through a change management workflow satisfying SOC 2 PI1.4.

4.7 Network Security Controls

All Azure Batch pools, Synapse Spark pools, and AML endpoints are deployed within a private virtual network with no public IP addresses. Service-to-service communication uses Azure Private Link. The API Management gateway is accessible only via Forte's internal network or site-to-site VPN. Network Security Groups restrict inter-service communication to defined routes, preventing lateral movement in the event of a compromised compute node.

5 Error Taxonomy and Expected Coverage

Table 2 summarizes the error classes addressed by each system layer.

Error Type	Layer 1	Layer 2
Year/date mismatch	✓	
Client name drift	✓	
Numeric inconsistency	✓	
Drug/indication drift	✓	
Temporal orphan	✓	
Semantic contradiction		✓
Claim/evidence mismatch		✓
Qualifier omission		✓
Rounding inconsistency	✓	✓
Layout-role mismatch	✓	

Table 2: Error taxonomy and layer coverage. Layout-role mismatch refers to entities whose structural position is inconsistent with their semantic role, detectable via LayoutLMv3 position encoding.

6 Discussion

6.1 Relationship to Behavioral Anomaly Detection

The architectural pattern proposed here — learning the distribution of normal configurations and flagging departures as anomalous — is isomorphic to the quantum behavioral containment framework proposed in [13]. The RBM's role as a probabilistic observer of document entity relationships maps directly onto the quantum probabilistic observer of AI behavioral state in the containment framework. In both cases, the system does not enumerate prohibited states

but learns what authorized or coherent states look like and identifies departures.

6.2 Limitations

The unsupervised component requires a training corpus of clean, verified documents representative of the organization's deliverable style. LayoutLMv3-large requires GPU compute for inference at acceptable latency, adding infrastructure cost relative to text-only extraction. The supervised layer requires labeled training data for the semantic contradiction task. The system does not address factual accuracy relative to external ground truth — a document that consistently asserts an incorrect market share figure across all slides will not be flagged.

6.3 Extension to Dual-Evaluator Scoring

An extension of this architecture incorporates a dual-evaluator pattern in which two independent model instances score each flagged content pair and disagreement between evaluators is itself treated as an anomaly signal [14]. This provides a second-order coherence signal independent of the individual model scores.

7 Conclusion

We have described a hybrid document coherence verification architecture combining layout-aware structural extraction via LayoutLMv3, unsupervised RBM anomaly detection over a semantic entity graph, and supervised semantic contradiction classification. The unsupervised RBM layer handles the majority of real-world coherence errors without requiring labeled error data. The supervised semantic contradiction layer handles the residual class of linguistically complex inconsistencies.

The architecture is deployable within enterprise Azure infrastructure using Azure Synapse Analytics for pipeline orchestration, Azure Batch for embarrassingly parallel corpus processing in plain Python, and Azure ML managed endpoints for open-source model serving. This combination satisfies SOC 2 Type II data governance requirements while avoiding the operational complexity of native PySpark development. Client document content remains within the organization's Azure tenant boundary at all times.

References

[1] Mann, W.C. and Thompson, S.A. (1988). Rhetorical structure theory: Toward a functional theory of text organization. *Text*, 8(3), 243–281.

[2] Lee, H., Peirsman, Y., Chang, A., Chambers, N., Surdeanu, M., and Jurafsky, D. (2011). Stanford's multi-pass sieve coreference resolution system at the CoNLL-2011 shared task. *Proceedings of CoNLL-2011*, 28–34.

[3] Cowie, J. and Lehnert, W. (1996). Information extraction. *Communications of the ACM*, 39(1), 80–91.

[4] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., and Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. *Proceedings of KDD 2020*, 1192–1200.

[5] Huang, Y., Lv, T., Cui, L., Lu, Y., and Wei, F. (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. *Proceedings of ACM MM 2022*, 4083–4091.

[6] Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. *Parallel Distributed Processing*, 1, 194–281.

[7] Hinton, G.E., Osindero, S., and Teh, Y.W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), 1527–1554.

[8] Fiore, U., Palmieri, F., Castiglione, A., and De Santis, A. (2013). Network anomaly detection with the restricted Boltzmann machine. *Neurocomputing*, 122, 13–23.

[9] Mihalcea, R. and Tarau, P. (2004). TextRank: Bringing order into texts. *Proceedings of EMNLP 2004*, 404–411.

[10] Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*.

[11] Ji, S., Pan, S., Cambria, E., Marttinen, P., and Yu, P.S. (2021). A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 494–514.

[12] Hinton, G.E. (2002). Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8), 1771–1800.

[13] Maskrey, M. (2026). Quantum behavioral containment: A probabilistic observer architecture for autonomous AI runtime monitoring. *Forte Analytics Research*, September 2026.

[14] Maskrey, M. (2024). Cooperative-adversarial dual evaluator architectures for structured document verification. *Unpublished manuscript*.